

Next

Documentação Técnica do Backend
Cloud Code (Parse Server) hospedado no Back4App

1 de outubro de 2026

Índice

1	Introdução	2
1.1	Objetivos	2
2	Arquitetura	2
2.1	Fluxo de uma requisição	2
3	Modelo de Dados	2
3.1	Classe <code>Vehicle</code>	3
3.2	Classe <code>Sale</code>	3
3.3	Classe <code>_User</code> (nativa, com campos extras)	3
4	Validações (<i>beforeSave</i>)	3
5	Cloud Functions	4
5.1	Autenticação	4
5.2	Veículos	4
5.3	Vendas	4
5.4	Usuários e painel	4
6	Segurança e Controle de Acesso	5
7	Implantação (Deploy)	5
8	Testes	5
9	Considerações Finais	6

1 Introdução

Este documento descreve a arquitetura, o modelo de dados e as funcionalidades do backend do sistema **Next**, responsável pela gestão de inventário de veículos, registro de vendas e controle de usuários de uma operação de venda de veículos seminovos.

O backend foi implementado como **Cloud Code** do Parse Server, hospedado na plataforma **Back4App** (<https://www.back4app.com>), substituindo a necessidade de manter um servidor dedicado. Toda a lógica de negócio é escrita em JavaScript e executada em funções remotas invocadas pelo frontend via SDK do Parse.

1.1 Objetivos

- Centralizar as regras de negócio do domínio (veículos, vendas e usuários) em um único ponto de execução no servidor.
- Validar dados antes de persisti-los no banco, evitando estados inconsistentes (ex.: preço negativo, status inválido).
- Expor uma API simples, baseada em chamadas remotas de função (*Cloud Functions*), consumida pelo frontend em Next.js.

2 Arquitetura

A arquitetura segue o modelo *Backend as a Service* (BaaS). O Back4App provê o Parse Server gerenciado (API REST/SDK, autenticação e banco de dados MongoDB); o projeto contribui apenas com o código de domínio, localizado em `cloud/main.js`.

Camada	Tecnologia	Responsabilidade
Cliente	Next.js (React)	Interface do usuário, consome o SDK do Parse
API	Parse Server (Back4App)	Autenticação, roteamento, ACL/CLP
Lógica de negócio	Cloud Code (<code>cloud/main.js</code>)	Validações e Cloud Functions
Persistência	MongoDB (gerenciado)	Armazenamento das classes do domínio

2.1 Fluxo de uma requisição

1. O frontend chama `Parse.Cloud.run("nomeDaFuncao", parâmetros)`.
2. O Parse Server recebe a requisição HTTPS e identifica a sessão do usuário autenticado (quando houver).
3. A função correspondente em `cloud/main.js` é executada, podendo disparar gatilhos `beforeSave` de validação antes de gravar qualquer objeto.
4. O resultado (JSON) é devolvido ao frontend.

3 Modelo de Dados

O domínio é composto por três classes persistidas no Parse: `Vehicle`, `Sale` e a classe nativa `_User` (estendida com campos próprios).

3.1 Classe Vehicle

Campo	Tipo	Descrição
marca	String	Fabricante do veículo
modelo	String	Nome do modelo
ano	Number	Ano de fabricação (entre 1990 e o ano atual + 1)
preco	Number	Preço de venda, deve ser maior que zero
tipo	String	Categoria (ex.: SUV, Picape, Sedã)
status	String	Disponível Vendido Reservado
tipoPreco	String	Condição de pagamento (ex.: "À vista")

3.2 Classe Sale

Campo	Tipo	Descrição
veiculo	Pointer<Vehicle>	Referência ao veículo vendido
consultor	String	Nome do responsável pela venda
valorFinal	Number	Valor efetivamente negociado
data	Date	Data da venda (preenchida automaticamente se omitida)

3.3 Classe _User (nativa, com campos extras)

Campo	Tipo	Descrição
nome	String	Nome completo do usuário
email	String	Também usado como <code>username</code>
role	String	Papel do usuário (padrão: <code>Consultor</code>)

4 Validações (*beforeSave*)

Antes de qualquer gravação, o Cloud Code valida os dados recebidos, impedindo que registros inconsistentes cheguem ao banco.

- **Vehicle:** marca, modelo e tipo obrigatórios; ano dentro do intervalo permitido; preço maior que zero; status restrito aos valores válidos; valores padrão aplicados quando omitidos (`status = "Disponível"`, `tipoPreco = "À vista"`).
- **Sale:** veículo e consultor obrigatórios; valor final maior que zero; data preenchida automaticamente quando ausente.
- **_User:** papel (`role`) padrão `"Consultor"` quando não informado no cadastro.

Listing 1: Validação da classe `Vehicle` (trecho de `cloud/main.js`)

```

1 Parse.Cloud.beforeSave("Vehicle", async (request) => {
2   validarVeiculo(request.object);
3 });

```

5 Cloud Functions

Todas as funções abaixo são invocadas via `Parse.Cloud.run("nome", parâmetros)` pelo frontend.

5.1 Autenticação

`registerUser(nome, email, senha, role?)`

Cria um novo `Parse.User`, define `username` e `email` como o e-mail informado, e atribui `role` (padrão "Consultor"). Retorna os dados públicos do usuário criado.

Login e logout não exigem Cloud Function própria: o frontend chama diretamente `Parse.User.logIn` e `Parse.User.logOut` do SDK.

5.2 Veículos

`createVehicle(marca, modelo, ano, preco, tipo, tipoPreco?, status?)` Requer usuário autenticado. Cria um novo veículo com ACL de leitura pública e escrita restrita à role `Administrador`.

`listVehicles(busca?, status?)` Lista veículos ordenados por data de criação (mais recentes primeiro). O parâmetro `busca` filtra por marca, modelo ou tipo (texto livre); `status` filtra pelo status exato.

`updateVehicleStatus(vehicleId, status)` Requer usuário autenticado. Atualiza o status de um veículo existente.

`deleteVehicle(vehicleId)` Requer usuário autenticado. Remove um veículo do inventário.

5.3 Vendas

`registerSale(vehicleId, consultor, valorFinal)` Requer usuário autenticado. Cria um registro de venda e marca o veículo correspondente como "Vendido".

`listSales()` Lista todas as vendas, com o veículo associado incluído (`include`), ordenadas da mais recente para a mais antiga.

5.4 Usuários e painel

`listUsers()` Requer usuário autenticado. Retorna nome, e-mail e papel de todos os usuários cadastrados.

`dashboardStats()` Calcula indicadores agregados: total de veículos, quantidade por status, número de vendas no mês corrente, receita do mês e taxa de conversão (percentual de veículos vendidos sobre o total do estoque).

Listing 2: Exemplo de Cloud Function (`registerSale`)

```
1 Parse.Cloud.define("registerSale", async (request) => {
2   if (!request.user) {
3     throw new Parse.Error(
```

```
4     Parse.Error.OPERATION_FORBIDDEN ,
5     "E preciso estar autenticado."
6   );
7 }
8 const { vehicleId, consultor, valorFinal } = request.params;
9
10 const vehicle = await new Parse.Query("Vehicle")
11   .get(vehicleId, { useMasterKey: true });
12
13 const sale = new (Parse.Object.extend("Sale"))();
14 sale.set("veiculo", vehicle);
15 sale.set("consultor", consultor);
16 sale.set("valorFinal", Number(valorFinal));
17 sale.set("data", new Date());
18 await sale.save(null, { useMasterKey: true });
19
20 vehicle.set("status", "Vendido");
21 await vehicle.save(null, { useMasterKey: true });
22
23 return sale;
24 });
```

6 Segurança e Controle de Acesso

- Funções que alteram dados (`createVehicle`, `updateVehicleStatus`, `deleteVehicle`, `registerSale`, `listUsers`) exigem `request.user` válido, ou seja, um usuário autenticado via sessão do Parse.
- O uso de `useMasterKey: true` dentro das Cloud Functions contorna ACLs/CLPs no servidor, mas a função em si já garante a checagem de autenticação antes de executar a operação — a Master Key nunca é exposta ao cliente.
- Veículos são criados com leitura pública (qualquer pessoa pode listar o estoque) e escrita restrita à *role* Administrador.

7 Implantação (Deploy)

1. Criar um aplicativo do tipo *Backend as a Service* no Back4App.
2. Copiar o conteúdo de `cloud/main.js` para o editor de Cloud Code do painel do Back4App (ou conectar um repositório Git).
3. Executar o *deploy* pelo próprio painel.
4. Validar o funcionamento chamando `dashboardStats` pelo **API Console** do Back4App.

8 Testes

Os testes funcionais das Cloud Functions são realizados manualmente pelo **API Console** do Back4App, utilizando requisições `POST` para o endpoint `functions/{nomeDaFuncao}`, com os parâmetros enviados como corpo JSON e a opção *Use Master Key* habilitada quando necessário.

9 Considerações Finais

A escolha por um backend serverless no Back4App reduziu a complexidade de infraestrutura do projeto, permitindo focar o esforço de desenvolvimento nas regras de negócio do domínio (inventário, vendas e usuários), mantendo uma API simples o suficiente para ser consumida diretamente pelo frontend em Next.js.